CS5998 Capstone Project

Submission 01 - Project Definition Document

Predicting Credit Default from Customer Statement Histories

Umaiyal S – 268507H
Repository: github.com/umaiyal-1208/CS5998-capstone-project

1. Problem Statement

Credit card default is one of the largest sources of loss for card issuers. Banks use default-risk scores to set credit limits, priorities collections and approve new lending. Most scoring systems in industry summarize a customer's recent statements into fixed numbers, such as the latest balance, average spend or count of late payments and feed these into a gradient boosted tree model. This summary step loses the order of events. A customer whose payments slip steadily over several months carries a different risk from one with a single bad month, even when their averages look alike.

This project asks whether that ordering holds useful signal. Using up to 13 monthly statements per customer, it predicts whether the customer will fail to pay the amount due within 120 days of their latest statement. The problem is framed as a binary classification task on tabular time-series data, with two linked goals:

- **(a)** build a reliable default-risk score from a customer's statement history, and
- **(b)** test whether deep sequence models (GRU, Transformer) that read the statements in order beat a strong gradient-boosting model on aggregated features, and at what cost in complexity and explainability.

2. Data Sources

Source	Description
Primary dataset	American Express – Default Prediction (Kaggle, 2022). Real, anonymised card data: about 5.5 million monthly statements for 458,913 labelled customers, with up to 13 statements each. There are 190 features in five groups: Delinquency, Spend, Payment, Balance and Risk. 11 of them are categorical. The target is 1 if the customer did not pay the amount due within 120 days of their latest statement. About 26% of customers default.
Working copy	The raw files are large (about 16 GB train, 33 GB test), so I will use the community integer-dtype Parquet version (about 5 GB). It holds the same information in a compact form that fits Colab.

Source	Description
Evaluation split	The competition's test labels were never released. All evaluations will therefore use the labelled training file, split by customer into 70% train, 15% validation and 15% test (stratified on the target). No customer appears in more than one split.
Consistency check	Row counts, customer counts and the default rate in the Parquet copy will be checked against a sample of the raw Kaggle CSV. The baseline score will also be compared with published public results for this dataset.

Sources: Kaggle competition, target definition, feature and class-balance figures, file sizes and metric

3. Intended Methods

- **Data engineering and scalable processing (CS5229 Big Data Analytics Technologies):** store the data in Parquet and use PySpark to build per-customer aggregates (last, mean, standard deviation, min, max, and change from first to last). Then build padded statement sequences with masks for customers who have fewer than 13 statements.
- **Exploratory analysis and preprocessing (CS5617 Data Science):** study missing value patterns (a missing field is often informative in credit data), encode categorical features, check how features drift over time, and run statistical tests when comparing models.
- **Pattern discovery (CS5803 Advanced Data Mining):** use UMAP projections and clustering to find behavioural segments of customers. Use Isolation Forest to flag unusual statement histories, and later check whether the models fail more often on these groups.
- **Baseline models (CS5621 Machine Learning, CS5802 Advanced Machine Learning):** logistic regression, then a LightGBM model tuned with Optuna, trained on the aggregated features. Class imbalance is handled with class weights, and I will compare this against resampling.
- **Improved models (CS5804 Advanced Deep Learning, CS5616 Natural Language Processing):** a GRU over the monthly statements, then a Transformer encoder that treats each statement like a token in a sentence, with learned embeddings for the categorical features. Dropout, layer normalisation and early stopping will control overfitting. The best sequence model will then be blended with LightGBM.
- **Explainability and uncertainty (CS5801 Advanced Artificial Intelligence):** SHAP for LightGBM, integrated gradients for the sequence model, and Monte Carlo dropout to estimate how confident each prediction is.

The work uses Python, PySpark, LightGBM, PyTorch, Optuna and SHAP on a free Colab or Kaggle GPU. No external LLM API is used in the pipeline.

4. Expected Outputs

- A reproducible data pipeline, from raw Parquet files to customer-level aggregates and statement sequences, with documented preprocessing and fixed random seeds.

- An exploration analysis covering missing value patterns, drift over time and behavioural customer segments.
- A baseline model (logistic regression and tuned LightGBM) and improved models (GRU, Transformer and a blended ensemble), all compared on the same held-out customer test set with confidence intervals.
- An ablation study showing how much statement history and each feature group contribute. This gives a clear answer to whether sequence modelling is worth its extra cost.
- An explainability and uncertainty analysis (SHAP, integrated gradients, Monte Carlo dropout) grouped by feature family.
- A public GitHub repository with a full pipeline and a steady, meaningful commit history.
- A lightweight scoring service: a FastAPI endpoint and a Streamlit dashboard showing a customer's default risk and the main factors behind it.

5. Key Risks and Assumptions

- **Model performance risk:** gradient boosting is very strong on tabular data, so the sequence models may not beat LightGBM. That outcome still answers the research question. The ablations and the ensemble will show where sequence information helps and where it doesn't.
- **Compute and memory risk:** 5.5 million statements can exceed free Colab memory. I will use the compact Parquet copy, float16 tensors and PySpark aggregation, prototype on a 10% customer sample, and checkpoint every training run.
- **Interpretability limitation:** the features are anonymized, so explanations stay at the level of feature groups (delinquency, spend, payment, balance, risk), not individual business variables.
- **Evaluation risk:** the competition's test labels are private. All results come from my own held-out customer split, with an optional single Kaggle submission as an outside check.
- **Data leakage risk:** splitting by customer, and fitting every preprocessing step on training data only, stops information from the test customers reaching the models.
- **Originally risk:** many public Kaggle notebooks exist for this dataset. All code will be my own, any borrowed idea will be cited, and regular commits will show how the work developed.
- **Assumption on labels and sampling:** the provider's default labels are assumed to be correct, and subsampling non-defaulters is assumed not to distort how customers are ranked.
- **Assumption of stability:** customer behavior within the data period is assumed to be reasonably stable. Wider economic shifts are outside the scope of this project, and I will check for drift using a time-aware split.
- **Fairness limitation:** the data has no protected attributes such as age or gender, so a formal fairness analysis is not possible. This will be reported as a limitation.